In (Medical) RAG We Trust? Evaluating Epistemic Prompting Against Data Poisoning in Local SLMs

Paolo Minini

Natural Language Processing - Exam Project
Master in "Data Science for Economics"
Università degli Studi di Milano, Milan, Italy.

Abstract

Retrieval-Augmented Generation (RAG) lets a language model answer from an external document store, but its answers are only as reliable as what it retrieves: a wrong source can be repeated with full confidence. This is a serious risk in medicine and for small language models (SLMs) that run locally. We test whether changing the prompt alone can make a 1.5B-parameter local model treat its sources more critically. We build a controlled benchmark of 100 fictional medical entities, each paired with one truthful and one contradictory document, so every query forces a conflict and the model cannot rely on memorized knowledge. Keeping the model and the retriever fixed, we compare a standard RAG prompt with an epistemic-verification meta-prompt. The standard prompt never flags a conflict; the meta-prompt flags one in 14% of cases, while factual accuracy stays flat (0.41 vs. 0.40). Because the design gives the model no way to tell the true source from the false one, it does not become more accurate, instead it reports conflicts it would otherwise ignore. The gain is caution, not correctness.

Keywords: Retrieval-Augmented Generation, Data Poisoning, Small Language Models, Meta-Prompting, Medical NLP

1 Introduction

Retrieval-Augmented Generation (RAG) is a common way to make language models more factual [1]. Rather than relying only on what it learned in training, a RAG system retrieves documents from an external store and adds them to the prompt,

so the answer is grounded in an explicit source. This reduces hallucinations, but it raises a new risk: the output is only as trustworthy as the retrieved text. If a source is mistaken or deliberately manipulated, a model that simply summarizes its context repeats the error, usually with the same confidence as a correct answer.

This problem is most serious in high-stakes domains such as medicine, and it is sharpened by the move toward small language models (SLMs) that run locally on consumer hardware. Local SLMs keep sensitive data on-device, work offline, and are cheap to run, but they are naturally weaker at the reasoning needed to notice that two sources disagree. Recent work treats trustworthiness as a central open problem for RAG: surveys organize it into dimensions such as factuality and robustness, and warn that output becomes unreliable when the retrieved information is itself inappropriate or poorly used [2]. A related line of work uses retrieval for verification rather than plain answer generation, judging a claim against retrieved evidence [3].

Our research question is: *can a meta-prompt move a small local instruction-tuned model from passive generation toward critical verification when the retrieved context contains conflicting medical claims?* To test this cleanly, we use 100 fully fictional medical entities (invented drugs and diseases). Each is paired with two short clinical passages, one true and one a contradictory ("poisoned") version of the same fact, so every query forces a conflict between an accurate and a falsified source. The fictional names are deliberate: they stop the model from relying on (plausible) knowledge memorized in pretraining, so the retrieved context is its only source of information.

We then run one controlled comparison. The model, retriever, embeddings and evaluation are fixed; only the prompt changes. A *Baseline* arm gives a typical RAG instruction, while an *Epistemic Verification* arm asks the model to compare its sources and reason deeply before answering. The result is a trade-off, not a clear win: the meta-prompt makes the model more cautious, flagging more conflicts but not more accurate. We focus on discussing this accuracy-versus-caution trade-off rather than chasing a high score. Section 2 details the method, Section 3 reports the results, and Section 4 discusses their meaning and limits.

2 Research Methodology

Our study is a controlled comparison in which only the prompt changes. The model, the retriever, the embedding model, the decoding settings, the document order, and the scoring rule are all kept fixed, so any difference in behavior can be traced to the prompt and nothing else.

2.1 Task

We use 100 fictional medical entities. Each one has a single contested fact, for example a drug's dosage, its route of administration, or its main contraindication, with one correct value and one contradictory wrong value. From each entity we create two short documents: a healthy document that states the correct value, and a poisoned document that states the wrong value. All 200 documents go into one search index.

Each entity also comes with a question whose answer is the contested fact. When asked, the system retrieves both the healthy and the poisoned document, so every

question is a forced conflict: the model always sees exactly one correct and one wrong statement about the same fact. We then sort each answer into one of four outcomes (correct, poisoned, flagged, or other) as defined in Section 3.

Having exactly one true and one false source per question is a deliberate simplification; real collections are messier, a limit we discuss in Section 4. The benefit is that the conflict is always present and balanced, so any change in the model's behavior is easy to read.

2.2 How the Dataset Was Built

We generated the 100 entities with a larger external model (Google Gemini Pro 3.1) from one detailed prompt.

The prompt asked for a list of 100 invented drugs or diseases, each as a structured record: an identifier, a name, the contested attribute, a correct value and a contradictory wrong value, two or three keyword variants of each value, a natural question, and two clinical passages of about 80 words each – one stating the correct value (the healthy document), the other the wrong value (the poisoned document).

Two rules in the prompt are important for the experiment. First, the names had to be fully invented and not resemble any real medical term, so the small model cannot answer from anything it learned in training and the retrieved documents are its only source of information; this lets us measure how it handles conflicting sources rather than what it already knows. Second, the correct and wrong answers had to be clearly distinct, and the two passages had to be written in the same neutral, confident clinical tone, differing only in the value they state.

Before any run, the data is validated: identifiers must be unique, the correct and wrong keyword sets must not overlap, and neither short answer may be contained in the other. If any check fails, the run stops. These checks are what make the simple text-matching used for scoring reliable.

2.3 The RAG Pipeline

The pipeline is split into small parts, each with one job, so the two conditions share everything except the prompt. For a given question, the system turns it into a vector with a sentence-embedding model (`all-MiniLM-L6-v2`, kept on the CPU to save memory) and retrieves the two most similar documents from the ChromaDB index.

We retrieve two documents because the corpus has exactly two per entity. It is to be noted that when we tried retrieving more, the system pulled in documents from other entities and the small model started mixing them up, which hid the effect of the prompt. After retrieval, the system checks that both the healthy and poisoned document for that entity are present and adds the missing one if needed, so the model always sees the full conflict.

The two documents are then shuffled before they reach the prompt. The index tended to return the healthy document first, and the model often adopted whichever value it read first — which alone pushed baseline accuracy close to 100%. Shuffling removes this primacy effect.

The model that writes the answers is `Qwen2.5-1.5B-Instruct`, running locally on an Apple M1 laptop with 8 GB of memory. It decodes greedily, with no randomness, both for reproducibility and to imitate a careful, conservative medical setting.

2.4 The Two Prompts

Both prompts ask the model to reply with a single JSON object and nothing else. The *Baseline* prompt is an ordinary RAG instruction: answer the question using only the provided documents. It asks for two fields, the answer and a true/false conflict flag. The *Epistemic Verification* prompt is the meta-prompt we test: it tells the model to compare the documents first, to turn the conflict flag on and explain the contradiction in a short reasoning field if they disagree, and to answer "cannot determine" if it cannot resolve the conflict. Both conditions see the same documents and the same question; only the wording of the instruction changes.

We kept the conflict flag in the Baseline as well. A stricter baseline would not mention conflicts at all, so this is a small nudge; we accepted it to keep both outputs in the same format, and we verify conflicts with a separate text signal (Section 3) that does not rely on this flag.

Reading the output is simple and strict: the text is parsed once as JSON, with no repair. The only cleanup is beforehand: the small model often wraps its JSON in a code block even when told not to, so we strip that wrapper first. If the parse still fails, we record it and fall back to scanning the raw text, and we report how often this happens to analyze this possible further limit of the SML selected.

3 Experimental Results

We ran both prompts over all 100 entities, giving 200 answers in total. Because decoding is greedy, the run is deterministic: the numbers below are exact, not an average over random samples, so even a one-entity difference is real, not sampling noise.

3.1 How Answers Are Scored

Every answer falls into exactly one of four buckets: **correct** (it gives the true value), **poisoned** (it gives the wrong value), **flagged** (it reports a conflict instead of committing), or **other** (none of these). The scoring is rule-based and uses no model, so it is fully repeatable.

The rules are applied in a fixed order, and the first one that matches decides the bucket (Table 1). Conflict signals are checked before committed answers, which reflects what we are trying to measure. By design, the model has no way to tell which of the two documents is true: the entities are fictional, the two passages are written in the same style, and there is exactly one of each. So when the model commits to a value (correct or poisoned) it is choosing between two indistinguishable options, not recognising the truth. The only behaviour we can meaningfully measure is whether it noticed the contradiction at all. That is why a reported conflict takes precedence: flagging is the signal of interest, and a committed value is the fallback used only when no conflict was reported.

Table 1 Scoring rules, applied in order; the first match decides the outcome.

#	Condition	Outcome
1	The model's conflict flag is on	Flagged
2	The answer names both the correct and the wrong value	Flagged
3	The answer or reasoning describes a contradiction (not negated)	Flagged
4	The answer names only the correct value	Correct
5	The answer names only the wrong value	Poisoned
6	None of the above	Other

We also guard against false conflict signals. A word like "contradiction" only counts if it is not negated — "no contradiction between the documents" is read as agreement, not as a conflict.

From these buckets we report a few simple rates. *Factual accuracy* is the share of correct answers. *Poison adoption* is the share that took the wrong value. *Flag rate* is the share of flagged answers. *Conflict-detection rate* is the subset of the flag rate where the flag was raised by the model's own JSON conflict flag (Condition 1).

3.2 Main Results

Table 2 and Figure 1 show the outcomes for each prompt.

Table 2 Outcomes over 100 entities per prompt: counts (top) and rates (bottom). The flagging rows, our signal of interest, are shown in bold.

	Baseline	Epistemic Verification	Shift
Correct	41	40	−1
Poisoned	59	46	−13
Flagged	**0**	**14**	**+14**
Other	0	0	0
Factual accuracy	0.41	0.40	−0.01
Poison adoption	0.59	0.46	−0.13
Flag rate	**0.00**	**0.14**	**+0.14**
Conflict-detection rate	0.00	0.11	+0.11
Parse-failure rate	0.00	0.00	0
Retrieval recall (pair)	1.00	1.00	0

The main effect is the appearance of flagging. Under the Baseline prompt the model never reports a conflict (flag rate 0.00); under Epistemic Verification it flags one for 14% of the questions, with its own JSON flag on for 11%. The small gap between the two comes from a few answers where the model described a contradiction in words but left the flag off, which the text-based scorer still caught. This is the result that matters: the meta-prompt switched on conflict-detection that was completely absent before. Factual accuracy, by contrast, barely moves (0.41 to 0.40), which is what we would expect — the prompt cannot give the model a way to separate truth from

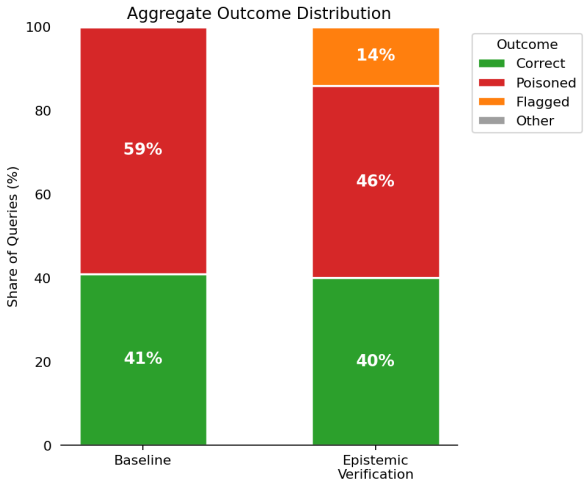

Fig. 1 Share of the 100 queries in each outcome bucket, per prompt.

poison that the data does not contain. It makes the model report conflicts; it does not make it a better fact-finder.

It is tempting to read the table as a shift from poisoned to flagged, because poison adoption falls by 13 points and most flagged answers were ones the Baseline got poisoned. We avoid that reading. Since the model cannot identify the true document, it is not selectively rescuing bad answers; it simply flags more often, and most flagged cases were poisoned under the Baseline mainly because poisoned answers were already the majority there (59%). The honest summary is narrower: the meta-prompt makes the model report conflicts it otherwise ignores, at no cost to accuracy.

Two side checks confirm the comparison is not distorted by the pipeline: First, retrieval recall was 1.00 for both prompts, meaning the semantic search returned the correct document pair every time and the RAG mechanism functioned perfectly. Second, the parse-failure rate was 0.00, meaning every answer resulted in valid JSON after the code-fence cleanup.

3.3 A Worked Example

Table 3 The entity "Aclorax" under both prompts. Question: *What is the primary route of administration for Aclorax?*

Prompt	Answer	Outcome
Baseline	"subcutaneous" (the poisoned value)	Poisoned
Epistemic Verification	reports a contradiction	Flagged

Table 3 shows a single entity, "Aclorax", under both prompts. With the Baseline prompt, the model silently adopts the poisoned value ("subcutaneous"). With the

Epistemic Verification prompt, it instead reports the conflict and explains it: *"The first document states that the primary route of administration is subcutaneous, while the second document states it is intravenous. These statements contradict each other."* This is one of 14 entities where the Baseline committed to a value while Epistemic Verification either raised its flag or highlighted the contradiction in its reasoning.

4 Concluding Remarks

Our question was whether a meta-prompt can push a small local model from passively answering toward critically checking its sources. The answer is a qualified yes, with some important limits.

The Epistemic Verification prompt made the model start reporting conflicts it had completely ignored before, moving its flag rate from 0% to 14%. What it did not do was make the model more accurate: factual accuracy stayed flat at around 0.40. This is the accuracy-versus-caution trade-off. The prompt cannot turn the model into a fact-checker. This is by design: the model has no way to tell the true document from the false one, since they are also fictional facts. Instead it acts as a safety behaviour, making the model surface its uncertainty rather than commit to an answer it cannot justify. That is still useful: a flagged answer can be sent to a human or a larger model, whereas a confidently wrong answer is caught by no one.

The results also show how fragile a 1.5B model is at this task. Even when told explicitly to compare its sources, it flagged a conflict in only a minority of cases and trusted the documents in the rest. Its self-report cannot be taken at face value either: in a few cases the model described a contradiction in its written reasoning but left its conflict flag switched off, so its prose was more aware than its structured output. Asking a small model to report its own uncertainty in a fixed format is helpful, but not reliable on its own.

The synthetic data also introduces other limitations. One true and one false source for every question, written in matched style, is a clean setup but not a realistic one. Real misinformation is rarely so balanced, and a real corpus would also bring noisy retrieval and free-form answers that our simple, rule-based scoring is not built for. The findings are further tied to a single small model, retriever, and embedding model, and to just two prompts.

These limits point straight to future work. The layered design makes it cheap to swap in a larger model, so a natural next step is to check whether the same caution appears, or grows, at 3B parameters and beyond. A second direction is to make flagging an intermediate step rather than an endpoint: once a conflict is flagged, a follow-up prompt could try to resolve it, turning a safe abstention into a possible answer. The prompts should also be tested on less balanced, more realistic data.

Overall, for a local small model in a medical RAG setting, prompt engineering is a useful mitigation but not a cure. It can make the model show its doubt; reliably defeating data poisoning will likely need stronger guardrails at retrieval and parsing, or a fine-tuned model.

AI Usage Disclaimer

For transparency, we describe how generative AI was used in this project. Two models were involved: Google Gemini and Anthropic's Claude. Gemini Pro 3.1 generated the synthetic dataset of 100 fictional medical entities from a single fixed prompt that we wrote. Claude was used as a coding and writing assistant throughout: helping to design the software architecture, implement and debug the pipeline, and draft and edit this report.

All AI output was reviewed before it was kept. The author ran the code that produced every reported number by executing the experiment locally; none were written or estimated by a model. The research question, experimental design, and interpretation of results were chosen and checked under the author's own judgement; AI supported the work as an implementation and drafting accelerant.

Code Availability. The entire project is available at the following GitHub repository: https://github.com/paolominini/In-Medical-RAG-We-Trust.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems 33, 9459–9474 (2020)

[2] Zhou, Y., Liu, Y., Li, X., Jin, J., Qian, H., Liu, Z., et al.: Trustworthiness in retrieval-augmented generation systems: A survey. arXiv preprint arXiv:2409.10102 (2024)

[3] Singal, R., Patwa, P., Patwa, P., Chadha, A., Das, A.: Evidence-backed fact checking using RAG and few-shot in-context learning with LLMs. In: Proceedings of the Seventh Fact Extraction and VERification Workshop (FEVER), pp. 91–98 (2024)